

# Deployment-Friendly Link Adaptation in Wireless Local-Area Network Based on On-Line Reinforcement Learning

Jie Chen<sup>ID</sup>, Juntao Ma<sup>ID</sup>, Yihao He<sup>ID</sup>, and Gang Wu<sup>ID</sup>, *Member, IEEE*

**Abstract**—In this letter, based on outer loop link adaptation (OLLA), we propose an adaptive OLLA algorithm (AOLLA), which calculates the OLLA offset adapted to the current wireless channel environment in real-time by utilizing frequency statistics of packet error within the last observation window. AOLLA algorithm contains numerous parameters that require manual tuning, which raises the deployment difficulty of the algorithm. An online reinforcement learning algorithm is designed to tune the parameters automatically and allow the AOLLA algorithm to rapidly deploy in different environments. We worked out an experimental validation by deploying the proposed algorithm on a software-defined radio hardware platform in three typical scenarios for deploying a wireless local-area network using IEEE 802.11ax standard with its original HESU packet format. Experimental results show that our designed algorithm has a 59.6% performance improvement compared to the original OLLA.

**Index Terms**—Wireless networks, fading channel, outer loop link adaptation, reinforcement learning, Q-learning.

## I. INTRODUCTION

IN THE process of wireless communication, the quality of the wireless signal is affected by small-scale fading, shadow fading, and large-scale fading. The mismatch between transmission parameters and wireless channel leads to severe link performance such as stalling in video transmission [1]. For a transmitter to utilize channel state information, there exists a fundamental issue, i.e., how to choose the appropriate modulation and coding scheme (MCS) to adapt to the wireless channel, which is known as link adaptation (LA). A deployment-friendly LA algorithm, which means that the algorithm can achieve fast adaptation to the channel state with low complexity in the presence of environment migration conditions, is the key to solving the challenge.

Nowadays, there have been some works on LA in terms of selecting MCS. Adaptive auto rate fallback (AARF) [2] and outer loop link adaptation (OLLA) [3] have been applied to the WiFi and 5G standards. In [4] the number of retransmissions is employed to calculate the signal-to-noise ratio (SNR) threshold. The authors in [5] added a starting offset to OLLA by the statistical results of the historical offset values, which accelerated the convergence of OLLA. In [6]

and [7], The authors proposed improvements to the OLLA algorithm to meet the URLLC requirements of 5G, including the use of mutual information and the number of LDPC decoder iterations to change the OLLA decision method. RL-based approaches have improved system performance in many communication scenarios [8], and RL-based approaches to optimize link adaption have recently received attention. In [9], the authors used Q-Learning to directly obtain the mapping from Channel Quality Indicator (CQI) to MCS. In [10] and [11], Thompson Sampling was utilized to estimate the probability distribution of the SNR, which led to more reliable SNR estimations. The authors in [12] employed deep reinforcement learning (DRL) to select the offset for the OLLA in real-time, while the deployment difficulty and arithmetic requirements were not considered.

To summarize the related work, the existing work does not sufficiently explore the relationship between wireless channel fading conditions and OLLA offset to improve link performance. Research work on RL adopted the mapping approaches of the action space to state space that had not demonstrated whether their algorithms can guarantee the original robustness after environment migration. Motivated by the above discussion, in this letter, we propose an adaptive OLLA (AOLLA) algorithm to calculate the offset of OLLA adaptively by introducing new channel information, frequency statistics of packet error within the last observation window, and an online RL algorithm is designed to train two Q-tables simultaneously to reduce the deployment difficulty of the algorithm. We implemented the proposed algorithm on a hardware platform using IEEE 802.11ax standard and validated the performance of the algorithm in different experimental environments to prove the high robustness performance. The proposed algorithm was demonstrated to achieve performance improvement by 59.6% compared to the original AARF and OLLA algorithms. The algorithm only requires the receiver to provide SNR estimation and CRC results. The complete source code implementing Q-learning-based AOLLA and related experiments is available at <https://github.com/owl-cj/Experimental-Guide-of-Deployment-friendly-RLLA.git>. This letter is organized as follows, Section II presents the problem formulation and the description of AARF and OLLA. Section III describes the proposed AOLLA algorithm and Q-Learning-based approach. Section IV presents the performance analysis and Section V summarizes the conclusions.

## II. PROBLEM FORMULATION AND STANDARD LINK ADAPTATION ALGORITHM

For the implementation of LA in a packet-switching-based wireless communication system, one of the most critical steps

Manuscript received 19 July 2023; revised 1 September 2023; accepted 20 October 2023. Date of publication 30 October 2023; date of current version 12 December 2023. This work was supported by the Fundamental Research Funds for the Central Universities under Grant 2242022k60006. The associate editor coordinating the review of this letter and approving it for publication was W. Jiang. (Corresponding author: Gang Wu.)

The authors are with the National Key Laboratory of Wireless Communications, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: 2439021231@qq.com; 15328790500@163.com; heyhao0@gmail.com; wugang99@uestc.edu.cn).

Digital Object Identifier 10.1109/LCOMM.2023.3327964

1558-2558 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.  
See <https://www.ieee.org/publications/rights/index.html> for more information.

is how to assemble channel information to predict the MCS adapted to the current wireless channel conditions so that a highly efficient transmission can be achieved. From [6], the goal of the LA algorithm can be modeled as:

$$m^*[t] = \arg \max_{m \in \mathcal{M}} \{ r_m[t] \mid PER_m[t] \leq PER_{target} \}, \quad (1)$$

where  $m$  denotes optional MCS from MCS space  $\mathcal{M}$ . The parameter  $r_m[t]$  denotes the largest supported data rate for MCS  $m$  and the  $t$ -th packet transmission.  $PER_{target}$  denotes the target packet error rate determined by the standard.  $PER_m[t]$  denotes the theoretical packet error rate for MCS  $m$  and the channel conditions of  $t$ -th packet transmission. This model is inner loop link adaptation (ILLA), which means the system obtains the SNR threshold lookup table for different MCSs to satisfy the  $PER_{target}$  requirement through offline link simulation with a pre-generated offline link model (OLM) in advance. However, in practice  $PER_m[t]$  is unknown. Firstly, the actual system is a causal system, which means that the SNR and other channel information we estimate are all related to the wireless transmissions that occurred in the past. The correct information for the next transmission remains a random variable and unknown. Secondly, in the process of wireless communication, the quality of the wireless signal is affected by small-scale fading, large-scale fading, and shadow fading simultaneously, and these channel fading conditions are time-varying and stochastic. It causes a time-varying and stochastic SNR, which is a major challenge that LA algorithms are required to dynamically adapt to wireless channels. Therefore AARF and OLLA were used to fulfill the requirement. AARF is the algorithm used in the IEEE 802.11 standard, which only requires feedback on the success or failure of this packet decoding to adjust the MCS in steps. OLLA firstly adds an offset on estimated SNR based on the result of packet decoding and secondly performs ILLA to select an MCS. The mentioned AARF and OLLA algorithms will be the baseline algorithm for comparison with the algorithm we designed.

### III. PROPOSED Q-LEARNING BASED LINK ADAPTATION

In this section, we first describe the AOLLA algorithm that we designed, which can more rapidly adapt to changes in the wireless channel for more efficient transmission compared to AARF and OLLA. It can be easily deployed in any packet-switched-based wireless communication system with low complexity. Then online RL is used to further optimize the AOLLA, using the Q-learning algorithm to improve the robustness of the AOLLA as well as the efficiency of the transmission.

#### A. Proposed AOLLA Link Adaptation Algorithm

AOLLA is evolved from OLLA, therefore, The receiver needs to estimate the SNR and compare it with the threshold vector of the OLM to select the MCS. Wireless channel exhibits strong time variability owing to its stochastic property. It causes a large variance in the SNR, which prevents the MCS selection method of the threshold vector from convergence to  $PER_{target}$ . OLLA algorithm gives an offset to the SNR estimation result by whether the CRC result is successful or

not, and this offset value is fixed. OLLA algorithm needs a long time to converge to reach the  $PER_{target}$ . AOLLA algorithm is designed to achieve dynamic adjustment of the offset value by introducing new information and can converge to  $PER_{target}$  more rapidly. The new information, packet error rate in the observation sliding window, is adopted, which is expressed as:

$$PER(n) = \begin{cases} \frac{N_F(n)}{n}, & \text{if } n \leq W \\ \frac{N_{Fw}(n)}{W}, & \text{if } W < n \leq N_t, \end{cases} \quad (2)$$

where  $n$  represents the number of transmissions that have already received and performed CRC.  $N_F(n)$  represents the number of received packets failing to pass the CRC.  $W$  stands for the length of the sliding window being the same unit as  $n$ .  $N_{Fw}$  represents the number of received packets failing to pass the CRC in the sliding window.  $PER(n)$  is updated each time a packet is received and a feedback packet is generated. With the introduction of new information, the algorithm has the ability to adjust the SNR threshold vector and retain  $PER_{target}$  under different wireless channel conditions. The algorithm is illustrated in Algorithm 1.

As described in Algorithm 1, the essential feature of the designed algorithm is that the threshold vector can be flexibly adjusted in terms of the error between targeted and realistic packet error rate  $e(i)$ . The following stages are essential to deploying the algorithm.

1) *Initialization*: For each packet received and performed CRC, the algorithm iterates once and outputs the required MCS of the next transmitted packet. In step 10, initial window length  $W_{initial}$  means there is an initialization phase in the designed algorithm. In this stage,  $PER(n)$  is not credible since  $n$  is small and we use the threshold comparison to calculate  $MCS$ . The value of the initial window is not supposed to be set too large. When  $n$  is greater than the initial window length, the first iteration is performed and an offset  $\Delta_{thr}(i)$  is calculated and added over SNR threshold vector  $\mathbf{SNR}_{thr}$  for each iteration to generate a new threshold vector  $\mathbf{SNR}_{thr}^{new}$ . Moreover, the method to calculate  $\widehat{SNR}$  we used is presented in the same URL of the source code.

2) *Critical Parameters*: It is necessary to make  $e(i)$  calculated in the cases of less than  $PER_{target}$  and greater than  $PER_{target}$  produce the same effect on the threshold offset calculation. Therefore,  $PER_{target}$  and  $PER(n)$  must be converted to the logarithmic domain to expand the value of the small packet error rate. Step 6 is to avoid infinite values shown in logarithmic calculations. The length of the sliding window  $W$  is vital to the performance of the algorithm. A greater value of  $W$  is more reliable for the calculation of the packet error rate, nevertheless, the rate of threshold adjustment can not keep up with the wireless channel change rate. On the contrary, a smaller value of  $W$  enables the speed of threshold adjustment to catch keep with the rapidly changing channel conditions, however, the low-resolution packet error rate calculation will lead to a bias that the final packet error rate statistical result will be larger than the targeted packet error rate.

3) *Adapting and Tracking to Wireless Channels*: SNR threshold offset  $\Delta_{thr}(i)$  is updated in terms of last offset

**Algorithm 1** AOLLA Algorithm

---

**Input:**  $\widehat{SNR}$ , the targeted packet error rate  
 $PER_{\text{target}}$ , the SNR threshold vector from simulation under assigned packet error rate  
 $SNR_{\text{thr}}$ , initial step size  $\mu(0) = 0.1$ , initial window length  $W_{\text{initial}}$

**Output:** the required MCS of the next transmitted packet  $MCS$

```

1  $PER_{\text{targetLog}} = 10 \log_{10}(PER_{\text{target}})$ ;
2  $\Delta_{\text{thr}}(0) = 0$ ;
3  $i = 1$ ;
4 for  $n = 1, 2, \dots, N_t$  do
5    $PER(n)$  is updated based on (2);
6   if  $PER(n) = 0$  then
7      $PER(n) = 0.01$ ;
8   end
9    $PER(n)_{\text{Log}} = 10 \log_{10} PER(n)$ ;
10  if  $n \leq W_{\text{initial}}$  then
11    Calculate  $MCS$  based on threshold comparison;
12  else
13    Calculate the error between targeted and realistic packet error rate:
14     $e(i) = PER(n)_{\text{Log}} - PER_{\text{targetLog}}$ ;
15    if  $e > 0$  then
16       $e(i) = \beta_1 e(i)$ ,  $\beta_1 > 1$ ;
17    else
18       $e(i) = \beta_2 e(i)$ ,  $0 < \beta_2 < 1$ ;
19    end
20     $\Delta_{\text{thr}}(i) = \Delta_{\text{thr}}(i-1) + \mu(i-1)e(i)$ ;
21     $\mu(i) = \alpha(\mu(i-1) + \gamma|e(i)|)$ ;
22     $i = i + 1$ ;
23     $SNR_{\text{thr}}^{\text{new}} = SNR_{\text{thr}} + \Delta_{\text{thr}}(i)$ ;
24    Calculate  $MCS$  based on threshold comparison with new SNR threshold vector  $SNR_{\text{thr}}^{\text{new}}$ ;
25  end

```

---

$\Delta_{\text{thr}}(i-1)$ ,  $e(i)$  and the variable step size  $\mu(i)$ . When the realistic packet rate is greater than the targeted packet error rate,  $e(i)$  will be greater than 0 as well as  $\Delta_{\text{thr}}(i)$  since  $\mu(i)$  is always a positive number, indicating that the current channel conditions are bad and suitable for a more conservative MCS judgment strategy.  $SNR_{\text{thr}}^{\text{new}}$  calculated in step 22 satisfies this requirement by adding a positive  $\Delta_{\text{thr}}(i)$  to make each threshold larger, which means that the same MCS can only be applied at a higher SNR. On the contrary, when the realistic packet rate is less than the targeted packet error rate,  $e(i)$  will be less than 0 as well as  $\Delta_{\text{thr}}(i)$ , indicating that the current channel conditions are good and suitable for a more radical MCS judgment strategy.  $SNR_{\text{thr}}^{\text{new}}$  satisfies this requirement by adding a negative  $\Delta_{\text{thr}}(i)$  to make each threshold smaller, which means that the same MCS can be employed at a lower SNR. The iteration of variable step size  $\mu(i)$  is based on the last step size  $\mu(i-1)$  and the absolute value of  $e(i)$ . The value of  $\alpha$  is quite small to maintain the previously calculated

offset  $\Delta_{\text{thr}}(i-1)$  if  $e(i)$  were close to zero, which means the current  $SNR_{\text{thr}}^{\text{new}}$  added by  $\Delta_{\text{thr}}(i)$  adapts to the current channel conditions and reaches the assigned packet error rate. When the absolute value of  $e(i)$  becomes larger, the step size  $\mu(i)$  increases and the effect on the MCS judgment becomes very obvious. The function of  $\gamma$  is to amplify the effect of  $e(i)$  to meet this requirement.

4) *Convergence Acceleration of the Algorithm*: Step 14 is designed to reinforce the response speed of the threshold adjustment while facilitating the maintenance of the  $\Delta_{\text{thr}}(i)$  when the SNR threshold vector is adapted to the current channel conditions. Multiplying the  $e(i)$  by a number  $\beta_1$  greater than 1 has the advantage of increasing the step size  $\mu(i)$  so that the value of the  $\Delta_{\text{thr}}(i)$  calculated this time varies significantly from the last value  $\Delta_{\text{thr}}(i-1)$ , while multiplying it by a number  $\beta_2$  less than 1 results in a small change in  $\Delta_{\text{thr}}(i)$ . As for  $\alpha$  and  $\gamma$ , they can be set to a fixed value and there is no need to change their values after the first deployment, even if the environment in which they are deployed changes. The reason we make this conclusion is that in a large number of experiments we have conducted, the values of  $\alpha$  and  $\gamma$  have remained unchanged.

Selecting the values of all parameters appearing in Algorithm 1 should follow a principle that the  $\Delta_{\text{thr}}(i)$  of each iteration can not be too large, otherwise, it would be difficult for the algorithm to converge. Finally, the MCS of the next transmitted packet is placed in the feedback packet. The transmitter will use the MCS stated in the feedback packet to send the next data packet.

**B. Q-Learning-Based AOLLA Algorithm**

The AOLLA algorithm contains numerous parameters that require manual tuning, which raises the deployment difficulty of the algorithm. Among them, the performance of the AOLLA is most sensitive to the effect of the  $PER$  scaling parameters  $\beta_1$  and  $\beta_2$ . Banach fixed-point theorem was employed in the proof of an explicit expression for the convergence of the OLLA in [13]. It can also be used on the proposed AOLLA and the first Banach fixed-point theorem condition is apparently fulfilled. However, it is very difficult to write an explicit expression that fulfills the second Banach fixed-point theorem condition. We address this problem using online RL. We adopt the Q-learning algorithm to find the most suitable  $\beta_1$  and  $\beta_2$  in the state space by training two Q-tables simultaneously to achieve automatic parameter tuning. It enhances the robustness of the algorithm and achieves a deployment-friendly link adaptive algorithm.

In the designed Q-Learning algorithm, the receiver is the agent which holds two action-value function tables, i.e. Q-table, for  $\beta_1$  and  $\beta_2$ . We define our state space  $\mathcal{S}_1$  and  $\mathcal{S}_2$  as the value of  $\beta_1$  and  $\beta_2$ . As required by Algorithm 1,  $\mathcal{S}_1$  takes on discrete real numbers greater than 1, and  $\mathcal{S}_2$  takes on discrete positive real numbers less than 1. The dimensions of  $\mathcal{S}_1$  and  $\mathcal{S}_2$  do not need to be kept the same, which is verified by the experiments in Section IV. The action space  $\mathcal{A}$  is defined as:

$$\mathcal{A} = \{a_0, 0, a_2\}, \quad (3)$$



**Algorithm 2** Q-Learning-Based AOLLA Algorithm

---

```

1 while  $W_q$ th transmission is completed do
2   Calculate reward  $r_t$  as (7);
3   Update  $Q^{\beta_1}(s_t, a_t)$  with state space  $S_1$  as (5);
4   Update  $Q^{\beta_2}(s_t, a_t)$  with state space  $S_2$  as (5);
5   if random number  $[0, 1] > \epsilon$  then
6     Select new action
7      $a_{t+1}^{\beta_1} = \arg \max_{a \in A} Q^{\beta_1}(s_{t+1}, a);$ 
8     Select new action
9      $a_{t+1}^{\beta_2} = \arg \max_{a \in A} Q^{\beta_2}(s_{t+1}, a);$ 
10    else
11      Select  $a_{t+1}^{\beta_1}$  and  $a_{t+1}^{\beta_2} \in A$  randomly.
12    end
13  Update state  $s_{t+1}^{\beta_1} \in S_1$  and  $s_{t+1}^{\beta_2} \in S_2$  as (4);
14   $s_{t+1}^{\beta_1}$  and  $s_{t+1}^{\beta_2}$  are the  $\beta_1$  and  $\beta_2$  in Algorithm 1
15  for next  $W_q$ th transmission;
16 end

```

---

where actions  $a_0 < 0 < a_2 \in \mathbb{Z}$  are the value by which the  $\beta_1$  and  $\beta_2$  increase or decrease each time the Q-tables are updated and the agent takes a new action. The action chosen by the agent will directly determine the state as:

$$s_{t+1} = s_t + a_t. \quad (4)$$

Q-tables will calculate the effective data rate during the iteration window  $W_q$  as the reward  $r$ , and will update the Q-table at the end of each  $W_q$ . It is calculated as:

$$Q^{new}(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_q(r_t + \gamma_q \max_{a \in A} Q(s_{t+1}, a) - Q(s_t, a_t)), \quad (5)$$

where  $0 < \alpha_q < 1$  is the learning rate and  $0 \leq \gamma_q \leq 1$  is a discount factor. At the initialization stage of the Q-learning algorithm, we first set all the values in the Q-tables to an infinite value, and for the first iteration of the action-value function, we replace the infinite value as follows:

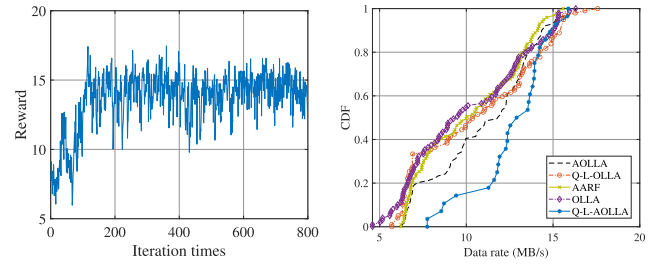
$$Q(s_t, a_t) = \frac{r_t}{1 - \gamma_q}. \quad (6)$$

The reward of the Q-learning is the effective throughput of the last iteration window, which is defined as:

$$r_t = \frac{N_{Byte}(t)}{T_{W_q}}, \quad (7)$$

where  $N_{Byte}(t)$  denotes the number of bytes received in each iteration window  $W_q$  without CRC failure. And  $T_{W_q}$  is the total transmission time in an iteration window  $W_q$ .

In summary, the proposed Q-learning-based AOLLA algorithm is illustrated in Algorithm 2. The epsilon greedy strategy is applied to balance exploration and exploitation. The value of  $\epsilon$  decreases linearly in each iteration until it falls to the target value,  $\epsilon_{target}$ .



(a) The Q-L-AOLLA learning curve. (b) CDF of data rate in simulations.

Fig. 1. Simulation results using IEEE 802.11ax HESU frame format.

#### IV. PERFORMANCE ANALYSIS AND COMPARISONS

Firstly, we show the learning curves of the proposed Q-learning algorithm (Q-L-AOLLA), which is trained by simulation using the HESU frame format of the IEEE 802.11ax protocol in a Model-D model of the TGax standard multipath channel and a shadow fading channel. The Q-L-AOLLA in the later experiments comparing the performance of multiple link adaptive algorithms through simulation is obtained using the results from this training. The Q-L-AOLLA learning curve graph is shown in Fig. 1a.

Afterward, we designed a simulation experiment. We transmit payload data and feedback packets using the HESU frame format of the IEEE 802.11ax protocol and the simulated channel contains large-scale fading, small-scale fading, and shadow fading. The SNR is set to 28 dB. 2500 transmissions are performed in one simulation. The large-scale fading is simulated by adding a sine curve with an amplitude of 6 dB to the signal-to-noise ratio, which undergoes two full periods in one simulation. Small-scale fading is realized by using the TGax standard multipath channel with the Model-D multipath model. Shadow fading is simulated by subtracting a random number with a variance of 1 dB from the SNR that follows a lognormal distribution. We obtained the cumulative distribution function (CDF) of the data rate of 5 LA algorithms shown in Fig. 1b to compare with state-of-art LA algorithms and an RL-based OLLA algorithm (Q-L-OLLA) [12]. The curves of CDF show that our designed AOLLA algorithm and Q-learning-based algorithm have significant improvement compared to the other algorithms.

A hardware platform was built to evaluate the performance of the proposed Q-Learning-Based AOLLA algorithm and to compare it with AARF and OLLA algorithms applied in the standard. We use two ADALM-PLUTO SDRs with SISO antennas controlled by two laptops respectively as transmitter and receiver. Matlab 2020 is required as a source code environment. The complete source code and other necessary information implementing Q-learning-based AOLLA and related experiments are available at <https://github.com/owl-cj/Experimental-Guide-of-Deployment-friendly-RLLA.git>. We use the same packet format as simulations. The MCS is decided on the receiver and reported to the transmitter through a feedback link. We perform the training phase of RL in the office environment with 25000 times packet transmissions, and then use the trained Q-tables straightforwardly in the office and two other new

TABLE I  
EXPERIMENTAL SETTINGS

|                                     |                     |                             |
|-------------------------------------|---------------------|-----------------------------|
| Experimental settings               | value               |                             |
| Number of transmitted packets       | 200                 |                             |
| Number of bytes in a packet         | 27648               |                             |
| Communication distance in office    | 5 m                 |                             |
| Communication distance in hallway   | 7 m                 |                             |
| Communication distance in classroom | 7 m                 |                             |
| LA algorithms settings              | variable            | value                       |
| AOLLA                               | $\alpha$            | 0.002                       |
|                                     | $W$                 | 8                           |
|                                     | $W_{initial}$       | 5                           |
|                                     | $\gamma$            | 3                           |
|                                     | $\beta_1$           | 1.75                        |
|                                     | $\beta_2$           | 0.65                        |
| Q-learning-based AOLLA              | $\mathbf{A}$        | $\{-0.2, 0.0, 0.2\}$        |
|                                     | $\mathbf{S}_1$      | $\{1, 2, 1.4, \dots, 2.4\}$ |
|                                     | $\mathbf{S}_2$      | $\{0.2, 0.4, 0.6, 0.8\}$    |
|                                     | $\gamma_q$          | 0.9                         |
|                                     | $\alpha_q$          | 0.1                         |
|                                     | $W_q$               | 25                          |
|                                     | $W$                 | 10                          |
|                                     | $\epsilon_{target}$ | 0.01                        |

TABLE II  
COMPARISON OF 5 LINK ADAPTIVE ALGORITHMS

| Algorithm                                | PER   | Data Rate (MB/s)per |
|------------------------------------------|-------|---------------------|
| office environment (Q-learning training) |       |                     |
| AARF                                     | 0.115 | 12.8428             |
| OLLA                                     | 0.085 | 13.2739             |
| Q-L-OLLA                                 | 0.09  | 13.1674             |
| Adaptive OLLA                            | 0.1   | 12.4337             |
| Q Learning Adaptive OLLA                 | 0.09  | 13.2749             |
| hallway environment                      |       |                     |
| AARF                                     | 0.125 | 8.8555              |
| OLLA                                     | 0.11  | 8.8513              |
| Q-L-OLLA                                 | 0.115 | 9.9588              |
| Adaptive OLLA                            | 0.11  | 11.2074             |
| Q Learning Adaptive OLLA                 | 0.115 | 12.1562             |
| classroom environment                    |       |                     |
| AARF                                     | 0.17  | 8.2944              |
| OLLA                                     | 0.125 | 7.6623              |
| Q-L-OLLA                                 | 0.11  | 8.7385              |
| Adaptive OLLA                            | 0.105 | 11.5029             |
| Q Learning Adaptive OLLA                 | 0.095 | 12.226              |

environments, hallway and classroom, to verify the robustness of the designed algorithms. More details about the experiment are listed in Table I.

A comparison of 5 LA algorithms is shown in Table II, and the environments of the experimental platform are listed in Table I. Data Rate is defined as the average data rate of correctly received bytes in total 200 times transmission. PER is the average packet error rate in total 200 times transmission. In terms of data rate performance, while AOLLA performed slightly poorly in the office environment compared to the AARF and OLLA, there was a significant improvement in performance in the hallway and classroom environments. The Q-learning-based AOLLA, on the other hand, has gained the best performance in all experimental environments (at most 59.6% improvement). In terms of the convergence performance of the PER, AARF did not converge in the classroom environment, while our proposed algorithms achieved a

convergence performance comparable to the performance of OLLA.

## V. CONCLUSION AND PERSPECTIVES

In this letter, we propose an AOLLA algorithm based on the OLLA algorithm for the link adaption problem in packet-switched-based wireless communication systems and design an online RL algorithm to optimize the tuning problem of AOLLA. We performed simulation experiments and deployed the algorithm on a hardware platform and evaluated the performance in three different environments. Simulation results show that our designed algorithm outperforms state-of-art algorithms and the previously proposed Q-learning OLLA algorithm [12]. The experimental results show that the performance of our proposed algorithms has a 59.6% performance improvement compared to the original OLLA. Future directions worth researching include designing different reward functions for the link adaptation algorithm to optimize user quality of experience and system energy consumption, etc.

## REFERENCES

- [1] H. Wang, L. Ji, and Z. Gao, "From PHY to QoE: A parameterized framework design," in *Proc. IEEE 95th Veh. Technol. Conf. (VTC-Spring)*, Jun. 2022, pp. 1–6.
- [2] M. Lacage, M. H. Manshaei, and T. Turletti, "IEEE 802.11 rate adaptation: A practical approach," in *Proc. ACM MSWiM*, Oct. 2004, pp. 126–134.
- [3] G. Pocovi, A. A. Esswie, and K. I. Pedersen, "Channel quality feedback enhancements for accurate URLLC link adaptation in 5G systems," in *Proc. IEEE 91st Veh. Technol. Conf. (VTC-Spring)*, May 2020, pp. 1–6.
- [4] D. Qiao, S. Choi, and K. G. Shin, "Goodput analysis and link adaptation for IEEE 802.11a wireless LANs," *IEEE Trans. Mobile Comput.*, vol. 1, no. 4, pp. 278–292, Oct. 2002.
- [5] A. Durán, M. Toril, F. Ruiz, and A. Mendo, "Self-optimization algorithm for outer loop link adaptation in LTE," *IEEE Commun. Lett.*, vol. 19, no. 11, pp. 2005–2008, Nov. 2015.
- [6] G. Pocovi, K. I. Pedersen, and P. Mogensen, "Joint link adaptation and scheduling for 5G ultra-reliable low-latency communications," *IEEE Access*, vol. 6, pp. 28912–28922, 2018.
- [7] E. Peralta, G. Pocovi, L. Kuru, K. Jayasinghe, and M. Valkama, "Outer loop link adaptation enhancements for ultra reliable low latency communications in 5G," in *Proc. IEEE 95th Veh. Technol. Conf. (VTC-Spring)*, Jun. 2022, pp. 1–7.
- [8] X. Lu et al., "Reinforcement learning based physical cross-layer security and privacy in 6G," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 425–466, 1st Quart., 2023.
- [9] M. P. Mota, D. C. Araujo, F. H. C. Neto, A. L. F. de Almeida, and F. R. Cavalcanti, "Adaptive modulation and coding based on reinforcement learning for 5G networks," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2019, pp. 1–6.
- [10] V. Saxena, H. Tullberg, and J. Jaldén, "Reinforcement learning for efficient and tuning-free link adaptation," *IEEE Trans. Wireless Commun.*, vol. 21, no. 2, pp. 768–780, Feb. 2022.
- [11] Y. Chen, H. Tang, J. Wang, and J. Song, "Adaptive link rate selection for throughput maximization with batched Thompson sampling," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, May 2022, pp. 1–6.
- [12] P. Kela, T. Höhne, T. Veijalainen, and H. Abdulrahman, "Reinforcement learning for delay sensitive uplink outer-loop link adaptation," in *Proc. Joint Eur. Conf. Netw. Commun. 6G Summit (EuCNC/6G Summit)*, Jun. 2022, pp. 59–64.
- [13] F. Blázquez-Casado, G. Gomez, M. D. C. Aguayo-Torres, and J. T. Entrambasaguas, "EOLLA: An enhanced outer loop link adaptation for cellular networks," *EURASIP J. Wireless Commun. Netw.*, vol. 2016, no. 1, pp. 1–16, Dec. 2016.